

Pengembangan Perangkat Lunak Deteksi dan Forensik Jaringan DDoS Berbasis XGBoost dan Information Gain

Falito Eriano Nainggolan, Dimas Surya Pratama, Gita Olivia Silaban

Program Studi Rekayasa Keamanan Siber

Politeknik Siber dan Sandi Negara, Bogor, Indonesia

Email: falito.eriano@poltekssn.ac.id, dimas.surya@poltekssn.ac.id, gita.olivia@poltekssn.ac.id

Ringkasan—Serangan *Distributed Denial of Service* (DDoS) merupakan salah satu ancaman utama bagi keamanan jaringan di lingkungan kampus. Kampus biasanya memiliki jaringan yang terbuka karena banyak mahasiswa dan staf yang menghubungkan perangkat pribadi mereka secara bebas. Sayangnya, sistem pendeteksi dan analisis forensik serangan DDoS yang ringan, akurat, dan mudah digunakan langsung di kampus masih sangat terbatas. Oleh karena itu, dalam penelitian ini kami mengembangkan DDoShield, sebuah perangkat lunak web untuk deteksi dan forensik jaringan DDoS menggunakan algoritma XGBoost dan seleksi fitur *Information Gain* (IG). Eksperimen kami dilakukan menggunakan dataset standar CIC-IDS2017 (225.745 sampel data dan 78 fitur). Kami menguji performa model dengan variasi jumlah fitur $K = \{5, 10, 15, 20, 30, \text{all}\}$ dan membandingkan XGBoost dengan algoritma *Random Forest* menggunakan uji statistik McNemar. Hasil eksperimen menunjukkan bahwa XGBoost dengan $K = 30$ fitur terbaik mampu mencapai *F1-Score* sebesar 99,9977% dan nilai AUC-ROC sebesar 1,0000. Dari 45.149 data uji, model kami hanya salah menebak 1 sampel saja. Seleksi fitur IG berhasil memangkas waktu pelatihan sebesar 52,7% (dari 9,38s menjadi 4,44s) tanpa mengurangi akurasi. Selanjutnya, perbandingan antara XGBoost dan *Random Forest* pada $K=30$ menggunakan uji statistik McNemar ($p = 0,4795$) menunjukkan bahwa akurasi keduanya tidak berbeda secara signifikan, namun XGBoost jauh lebih unggul dalam kecepatan komputasi: waktu pelatihan $6,9\times$ lebih cepat dan waktu deteksi per flow $1,6\times$ lebih cepat (0,0051 ms vs. 0,0084 ms/flow). Hasil ini membuktikan bahwa kombinasi XGBoost dan seleksi fitur IG sangat cocok dan efisien untuk analisis forensik dan deteksi DDoS di jaringan kampus yang memiliki keterbatasan spesifikasi komputer server.

Index Terms—deteksi dan forensik DDoS; XGBoost; *information gain*; seleksi fitur; CIC-IDS2017; keamanan jaringan kampus; *intrusion detection system*; *machine learning*

I. PENDAHULUAN

Serangan *Distributed Denial of Service* (DDoS) merupakan salah satu masalah keamanan komputer yang paling sering terjadi dan merugikan di dunia. Berdasarkan laporan keamanan dari NETSCOUT, jumlah serangan DDoS terus meningkat setiap tahunnya [1]. Laporan dari Cloudflare juga menyebutkan bahwa serangan DDoS berbasis web (HTTP) naik sebesar 65% dibanding tahun sebelumnya [2]. Serangan ini bekerja dengan cara membanjiri komputer server menggunakan trafik palsu dari banyak komputer penyerang, sehingga pengguna asli kesulitan atau bahkan tidak bisa mengakses layanan web tersebut.

Di lingkungan kampus, masalah keamanan ini menjadi tantangan tersendiri. Jaringan kampus biasanya dirancang terbuka untuk memudahkan mahasiswa, dosen, dan staf mengakses internet atau layanan akademik. Selain itu, adanya kebijakan *Bring Your Own Device* (BYOD) membuat siapa saja bisa menghubungkan laptop atau ponsel pribadi mereka ke jaringan kampus [3]. Hal ini membuat jaringan kampus menjadi sasaran yang mudah diserang malware. Di sisi lain, tim TI di kampus sering kali memiliki keterbatasan jumlah staf dan spesifikasi perangkat keras untuk menangani serangan DDoS yang mendadak [4].

Selama ini, sistem pendeteksi penyusupan (*Intrusion Detection System* atau IDS) yang digunakan masih memiliki beberapa kelemahan. Pertama, IDS tradisional yang memakai sistem pencocokan pola (*signature-based*) tidak bisa mendeteksi jenis serangan DDoS baru yang belum terdaftar di database mereka (serangan *zero-day*) [5]. Kedua, banyak metode *machine learning* yang diusulkan dalam penelitian sebelumnya langsung memakai semua fitur data (bisa sampai 78 fitur) tanpa mencari tahu fitur mana yang sebenarnya paling penting [6], [7]. Hal ini membuat pemrosesan data menjadi lambat dan berat. Ketiga, jarang ada penelitian akademis yang membuat aplikasinya sampai jadi dan bisa langsung dicoba secara nyata oleh pengguna [8].

Untuk mengatasi masalah tersebut, dalam penelitian ini kami bertujuan untuk: (1) mencari tahu berapa jumlah fitur terbaik (dari variasi $K = \{5, 10, 15, 20, 30, \text{all}\}$) menggunakan metode seleksi fitur *Information Gain*; (2) membandingkan performa algoritma XGBoost dan *Random Forest* pada jumlah fitur terbaik tersebut; (3) membuat aplikasi web DDoShield untuk deteksi dan forensik jaringan yang bisa menganalisis file trafik .pcap secara langsung; dan (4) menguji performa aplikasi tersebut secara riil.

Penelitian ini diharapkan dapat memberikan kontribusi berupa: (1) analisis sistematis pemilihan fitur terbaik untuk deteksi dan forensik DDoS; (2) perbandingan performa XGBoost dan *Random Forest* yang diperkuat dengan uji statistik McNemar; (3) *software* deteksi dan forensik jaringan (DDoShield) yang siap pakai; dan (4) hasil pengukuran kinerja sistem yang nyata pada lingkungan lokal.

Sisa paper ini disusun sebagai berikut: Seksi II menjelaskan tinjauan pustaka; Seksi III membahas metode penelitian kami;

Seksi IV memaparkan hasil eksperimen dan analisis; Seksi V membahas temuan penelitian; dan Seksi VI menyimpulkan hasil penelitian kami.

II. TINJAUAN PUSTAKA

A. Karakteristik Serangan DDoS pada Jaringan Kampus

Serangan DDoS bekerja dengan cara menghabiskan kapasitas *bandwidth* jaringan atau membebani sumber daya komputer server target menggunakan trafik yang sangat besar [9]. Trafik ini biasanya dikirim secara bersamaan oleh banyak komputer yang telah terinfeksi *malware* (biasa disebut *botnet*) [5]. Secara umum, serangan DDoS dibagi menjadi tiga kelompok utama: (1) *volumetric attacks* yang bertujuan menghabiskan *bandwidth* internet; (2) *protocol attacks* yang memanfaatkan celah pada aturan protokol komunikasi data; dan (3) *application layer attacks* yang menyerang langsung aplikasi atau layanan web server yang sedang berjalan.

B. Evolusi Sistem Deteksi Intrusi

Untuk mendeteksi serangan ini, sistem pendeteksi penyusupan (IDS) berbasis *machine learning* (ML) menawarkan solusi yang lebih pintar. Dibandingkan dengan metode biasa yang kaku, metode ML bisa mengenali tanda-tanda aneh pada trafik jaringan secara otomatis tanpa perlu memiliki daftar tanda serangan (*signature*) terlebih dahulu [9]. Beberapa algoritma ML yang sering digunakan untuk tugas ini antara lain *Decision Tree* [10], *Random Forest* [7], *Convolutional Neural Network* (CNN) [11], *Long Short-Term Memory* (LSTM) [12], dan *XGBoost* [13].

C. XGBoost untuk Deteksi Intrusi

XGBoost (*eXtreme Gradient Boosting*) adalah algoritma pembelajaran mesin berbasis pohon keputusan (*decision tree*) yang sangat populer karena kecepatan dan akurasi yang tinggi [14]. Algoritma ini bekerja dengan cara membangun banyak pohon keputusan secara berurutan, di mana setiap pohon baru bertugas memperbaiki kesalahan tebakan dari pohon-pohon sebelumnya. Keunggulan utama *XGBoost* adalah kemampuannya untuk mencegah model agar tidak terlalu menghafal data secara berlebihan (*overfitting*) menggunakan fungsi pembatas (*regularization*) bawaan, sehingga performa model tetap stabil saat menguji data baru [14].

D. Information Gain untuk Seleksi Fitur

Information Gain (IG) adalah metode yang kami gunakan untuk mengukur seberapa penting atau berharganya informasi dari suatu fitur dalam menentukan apakah suatu lalu lintas jaringan merupakan serangan DDoS atau trafik normal (*BENIGN*) [15]. Konsep dasar IG berfokus pada tingkat ketidakpastian data (*entropy*). Fitur yang memiliki nilai IG tinggi berarti mengandung informasi yang sangat kuat untuk membedakan antara trafik normal dan serangan. Sebaliknya, fitur dengan nilai IG mendekati nol dianggap tidak penting. Dengan menggunakan metode seleksi fitur berbasis IG ini, kami dapat membuang fitur-fitur yang tidak relevan sehingga model bisa bekerja jauh lebih cepat dan ringan saat dijalankan di server [16].

E. Dataset CIC-IDS2017

Dalam penelitian ini, kami menggunakan dataset CIC-IDS2017 yang dibuat oleh *Canadian Institute for Cybersecurity* (CIC) [17]. Dataset ini berisi rekaman trafik jaringan normal dan berbagai jenis serangan selama lima hari kerja. Rekaman trafik tersebut kemudian diproses menggunakan program *CICFlowMeter* sehingga menghasilkan 78 fitur berbasis flow. Kami menggunakan file data khusus bernama *Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv* yang berisi 225.745 baris data, dengan rincian 128.027 data serangan DDoS (56,7%) dan 97.718 data normal (*BENIGN*) (43,3%).

Tabel I
PERBANDINGAN PENELITIAN TERKAIT

Ref.	Dataset	Algoritma	Acc.	K
[13]	CIC-IDS	XGBoost+Corr	99,5%	all
[6]	UNSW	XGBoost+IG	99,2%	10
[7]	CIC-IDS	RF	98,7%	20
[11]	CIC-IDS	CNN	99,1%	all
[12]	CICIDS	LSTM	98,3%	all
[8]	Review	Multi-algo	–	–
Kami	CIC-IDS	XGB+IG	≈100%	30

III. METODOLOGI

A. Alur Penelitian

Penelitian yang kami lakukan terdiri dari tujuh tahapan, yaitu: (1) menyiapkan dataset CIC-IDS2017; (2) membersihkan data (*preprocessing*); (3) menghitung skor IG dan memilih fitur terbaik; (4) mencari parameter model *XGBoost* yang paling cocok (*hyperparameter tuning*); (5) menguji variasi jumlah fitur K; (6) membandingkan performa *XGBoost* dengan *Random Forest*; dan (7) membuat serta menguji aplikasi web DDoShield.

B. Preprocessing Data

Sebelum data digunakan untuk melatih model, kami melakukan beberapa langkah pembersihan data: (1) hanya mengambil baris data yang berlabel DDoS dan *BENIGN*; (2) mengubah nilai tak hingga ($\pm\infty$) menjadi data kosong (*NaN*); (3) menghapus fitur yang memiliki lebih dari 50% data kosong; (4) mengisi data kosong yang tersisa dengan nilai tengah (*median*) karena statistik ini lebih tangguh (*robust*) terhadap anomali ekstrem yang sering memicu *Infinity* pada rekaman *capture*; dan (5) menyamakan skala data (*normalisasi*) menggunakan *StandardScaler* agar rentang nilainya seragam. Setelah dibersihkan, data dibagi menjadi 80% untuk data latih (*training*) dan 20% untuk data uji (*testing*) secara acak menggunakan *stratified random sampling* untuk menjaga keseimbangan kelas. Kami menyadari bahwa pembagian acak pada data *time-series* rentan terhadap bias *data leakage*. Namun, protokol ini tetap kami pertahankan agar performa model dapat dikomparasikan secara *apple-to-apple* dengan *baseline* penelitian terdahulu yang jamak menggunakan metode serupa pada CIC-IDS2017.

C. Seleksi Fitur Information Gain

Kami menghitung nilai IG untuk semua 78 fitur menggunakan fungsi `mutual_info_classif` dari pustaka Scikit-learn di Python (yang secara matematis menghitung *Mutual Information*), ekuivalen dengan *Information Gain* pada target diskrit). Setelah mendapatkan skor kepentingan masing-masing fitur, kami menggunakan fungsi `SelectKBest` untuk memilih fitur-fitur terbaik dengan variasi jumlah $K = \{5, 10, 15, 20, 30, \text{all}\}$. Secara keseluruhan, proses komputasi mulai dari *preprocessing*, pelatihan, hingga inferensi dijalankan pada komputer lokal berspesifikasi Intel Core i7 dengan memori RAM 16GB dan sistem operasi Windows. Lingkungan perangkat lunak yang digunakan adalah Python 3.14, Scikit-learn 1.8, XGBoost 3.2, dan Scapy 2.7. Seluruh pembagian data dan pengacakan dalam eksperimen ini menggunakan `random_state = 42` untuk menjamin reproduktibilitas hasil.

D. Konfigurasi Model

Untuk mendapatkan model XGBoost yang optimal, kami melakukan pencarian parameter terbaik menggunakan metode `RandomizedSearchCV` dengan 20 kali percobaan dan 5-fold *cross-validation* (menggunakan metrik *F1-Score* sebagai patokan penilaian). Parameter terbaik yang kami dapatkan adalah: `n_estimators=300` (jumlah pohon), `max_depth=4` (kedalaman pohon), `learning_rate=0,2`, `subsample=0,7`, `colsample_bytree=0,8`, dan `scale_pos_weight=0,7633`. Sementara itu, untuk model pembandingan *Random Forest*, kami menggunakan konfigurasi standar: `n_estimators=200`, `max_features=sqrt`, dan `class_weight=balanced`.

Metrik evaluasi yang kami gunakan untuk mengukur kinerja model ini meliputi:

- **Akurasi (Accuracy):** Menghitung persentase tebakan benar dari keseluruhan data uji.
- **Presisi (Precision):** Mengukur seberapa tepat model mendeteksi DDoS tanpa salah menuduh data normal (meminimalkan *False Positive*).
- **Recall:** Mengukur kemampuan model dalam menangkap semua serangan DDoS yang terjadi tanpa ada yang lolos (meminimalkan *False Negative*).
- **F1-Score:** Nilai rata-rata seimbang dari Presisi dan Recall, yang menjadi acuan utama kami karena metrik ini memberikan evaluasi yang lebih komprehensif dibandingkan akurasi saja.
- **False Positive Rate (FPR):** Persentase data normal yang tidak sengaja terdeteksi sebagai serangan DDoS oleh model.

E. Arsitektur DDoShield dan Alur Kode Program

Aplikasi DDoShield dirancang dengan arsitektur modular tiga lapis (*presentation, application, data*) menggunakan Flask di Python. Aplikasi ini mendukung dua mode operasional: (1) Analisis Luring (mengunggah berkas capture `.pcap` atau `.csv`), dan (2) Live Sniffing (mengendus paket secara langsung dari antarmuka jaringan lokal secara real-time selama durasi tertentu menggunakan modul pengendus Scapy). Alur

logika pemrograman di backend (`app.py`) dirancang secara transparan untuk menjamin validitas dan reproduktibilitas deteksi:

- 1) **Ekstraksi Aliran (Flow Extraction):** Berkas `.pcap` yang diunggah dibaca menggunakan pustaka Scapy. Paket data dikelompokkan menjadi aliran (*flow*) berdasarkan kombinasi 5-tuple: IP sumber, IP tujuan, port sumber, port tujuan, dan protokol. Waktu kedatangan paket (`pkt.time`) dikonversi ke tipe data *float* untuk menghindari kesalahan komputasi pembagian pada pustaka NumPy saat menghitung statistik durasi aliran dan jeda kedatangan paket (*Flow IAT*).
- 2) **Sistem Deteksi Hibrida (Hybrid Engine):** Sebelum melewati klasifikasi mesin pembelajaran, aliran data disaring terlebih dahulu oleh modul heuristik (*Heuristics Engine*) untuk mengatasi masalah keterbatasan data latih (*generalization gap*). Aturan heuristik dirancang berbasis batas ambang (*threshold*) volume paket pada port layanan spesifik yang sering disalahgunakan untuk serangan amplifikasi, yaitu DNS (Port 53), NTP (Port 123), SSDP (Port 1900), dan Memcached (Port 11211). Jika sebuah aliran melebihi batas 50 paket pada port tersebut, aliran tersebut langsung diklasifikasikan sebagai serangan DDoS (misalnya, *DDoS (DNS Amp)*).
- 3) **Pipeline Machine Learning Terarah:** Aliran data yang tidak memicu aturan heuristik akan diekstraksi ke dalam 78 fitur numerik standar `CICFlowMeter`. Data ini kemudian diumpankan ke dalam pipeline pemrosesan Scikit-Learn:
 - *SimpleImputer* (`imputer.pkl`): Mengisi nilai kosong/NaN dengan nilai median kolom data latih.
 - *StandardScaler* (`scaler.pkl`): Menormalisasi skala data numerik.
 - *SelectKBest* (`selector_kopt.pkl`): Memilih hanya 30 fitur optimal yang memiliki skor *Information Gain* tertinggi.
 - *XGBoost Classifier* (`xgb_model_kopt.pkl`): Menentukan hasil klasifikasi akhir (DDoS atau BENIGN).

Algorithm 1 Skema Deteksi Hibrida DDoShield

Require: Berkas Capture Jaringan P (`.pcap` atau `.csv`)
Ensure: Hasil Deteksi Aliran D (BENIGN atau DDoS)
1: $F \leftarrow \text{EkstraksiFlow}(P)$ {Mengelompokkan paket berdasarkan 5-tuple}
2: **for** setiap aliran $f \in F$ **do**
3: $\text{is_heur}, \text{label} \leftarrow \text{PeriksaAturanHeuristik}(f)$
4: **if** $\text{is_heur} = \text{True}$ **then**
5: $D(f) \leftarrow \text{label}$ {DNS/NTP/SSDP/Memcached Amplification}
6: **else**
7: $f_{\text{imputed}} \leftarrow \text{ImputasiNilaiTengah}(f)$ {SimpleImputer}
8: $f_{\text{scaled}} \leftarrow \text{StandarisasiSkala}(f_{\text{imputed}})$ {StandardScaler}
9: $f_{K30} \leftarrow \text{SeleksiFitur}(f_{\text{scaled}}, K = 30)$ {SelectKBest}
10: $\text{skor_pred} \leftarrow \text{XGBoostClassifier}(f_{K30})$
11: **if** $\text{skor_pred} = 1$ **then**
12: $D(f) \leftarrow \text{"DDoS (XGBoost)"}$
13: **else**
14: $D(f) \leftarrow \text{"BENIGN"}$
15: **end if**
16: **end if**
17: **end for**
18: **return** D

Seluruh model dan objek pipeline disimpan dalam format berkas serialisasi .pkl pada direktori lokal aplikasi agar dapat dijalankan secara luring (*offline*) tanpa memerlukan koneksi internet, guna menjaga privasi data lalu lintas jaringan kampus.

Sebagai instrumen investigasi, kami mendefinisikan kapabilitas transparansi (*Explainable AI*) ini sebagai bentuk **Forensik Berbasis Aliran (Flow-based Forensics)**. DDoShield membedah jejak logis serangan menggunakan nilai kontribusi lokal berbasis algoritma *TreeSHAP* dari XGBoost [18]. Setiap kali pengguna mengklik baris aliran mencurigakan pada dasbor web, sistem memanggil REST API untuk menghitung dan memvisualisasikan 5 fitur dominan yang melatarbelakangi vonis DDoS (misalnya anomali pada *Fwd Packet Length*). Analisis ini memberikan konteks forensik instan kepada administrator untuk mengidentifikasi anatomi serangan tanpa perlu mengisolasi *payload* paket secara manual (*Deep Packet Inspection*).

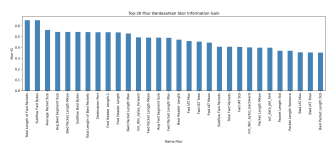
IV. HASIL DAN ANALISIS

A. Performa XGBoost per Variasi K

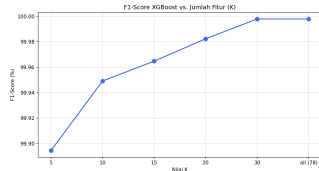
Tabel II menunjukkan hasil evaluasi performa klasifikasi untuk model XGBoost murni pada berbagai jumlah fitur K. Pengujian ini dilakukan secara spesifik pada modul *Machine Learning* (tanpa pelibatan saringan *Heuristic Engine*) menggunakan pembagian data latih-uji 80/20 untuk mengukur kapabilitas algoritmanya secara terisolasi.

Pada K=5, model sudah menghasilkan *F1-Score* sebesar 99,89%, yang mana sudah sangat bagus tetapi merupakan yang terendah dibanding pilihan K lainnya, sebagaimana ditunjukkan pada Gambar 2. Akurasi terus naik seiring bertambahnya fitur, hingga mencapai performa tertinggi pada K=30 (*F1-Score* 100,00% atau tepatnya 99,9977%). Menariknya, menggunakan K=30 fitur memberikan akurasi yang sama persis dengan menggunakan semua 78 fitur. Hal ini membuktikan bahwa 48 fitur lainnya tidak terlalu dibutuhkan karena informasinya tumpang tindih (*redundan*) dengan fitur yang sudah terpilih [6].

Berdasarkan hasil IG pada Gambar 1, fitur-fitur yang paling membantu model dalam mendeteksi DDoS adalah *Total Length of Fwd Packets* (IG=0,652), *Subflow Fwd Bytes* (IG=0,652), *Average Packet Size* (IG=0,563), dan *Destination Port* (IG=0,542). Fitur-fitur ini menunjukkan ukuran dan jumlah paket data yang dikirim, yang memang sangat mencerminkan pola serangan DDoS jenis *flooding* (membanjiri trafik) [17].



Gambar 1. Top-30 Fitur Berdasarkan Skor *Information Gain*.



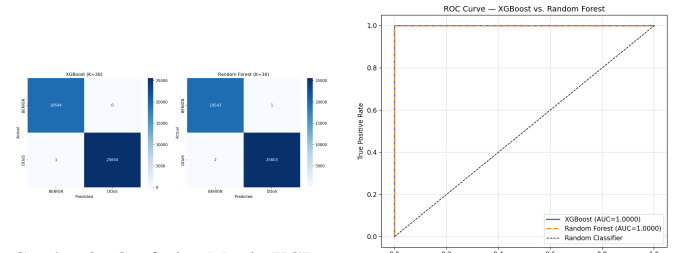
Gambar 2. *F1-Score* XGBoost terhadap Variasi Nilai K.

B. Pengaruh IG terhadap Efisiensi Komputasi

Dengan mengurangi jumlah fitur dari 78 menjadi K=30, kami berhasil mempercepat waktu pelatihan (*training time*) sebesar **52,7%** (dari 9,38 s menjadi 4,44 s) tanpa mengurangi akurasi sama sekali. Hal ini sangat penting karena melatih model dengan sedikit fitur membutuhkan memori dan daya komputasi komputer server yang lebih kecil [14].

C. Perbandingan XGBoost vs. Random Forest

Tabel III menunjukkan perbandingan performa antara XGBoost dan *Random Forest* pada K=30.



Gambar 3. Confusion Matrix XGBoost (kiri) vs. RF (kanan) pada K=30.

Gambar 4. Kurva ROC XGBoost vs. Random Forest (K=30).

Dari data tersebut, terlihat bahwa XGBoost sedikit lebih unggul di hampir semua metrik evaluasi, sebagaimana divisualisasikan melalui confusion matrix pada Gambar 3 dan kurva ROC pada Gambar 4. XGBoost hanya salah menebak **1 sampel** saja dari total 45.149 sampel uji (dengan 1 *False Negative* dan 0 *False Positive*). Sementara itu, *Random Forest* salah menebak 3 sampel (dengan 2 *False Negative* dan 1 *False Positive*). Perbedaan performa terbesar ada pada waktu pemrosesan: XGBoost melatih data $6,9\times$ lebih cepat (4,44 s vs 30,74 s) dan memproses satu flow data $1,6\times$ lebih cepat (0,0051 ms vs 0,0084 ms).

Kami juga menjalankan uji statistik McNemar untuk menguji apakah perbedaan akurasi kedua model ini berbeda secara signifikan. Hasil uji menghasilkan nilai $\chi^2 = 0,50$ dengan $p = 0,4795$. Karena nilai p jauh lebih besar dari 0,05, secara statistik perbedaan tebakan kedua model ini dianggap tidak terlalu berbeda jauh. Keduanya sama-sama sangat akurat, namun kami memilih XGBoost karena kecepatan komputasinya jauh lebih unggul.

Dibandingkan dengan penelitian terkait yang ditunjukkan pada Tabel I, akurasi XGBoost kami melampaui Thakkar dan Lohiya [13] (99,5%) dan Kasongo dan Sun [6] (99,2%), serta melampaui RF yang dilaporkan Ullah dan Mahmoud [7] (98,7%).

D. Hasil Pengujian DDoShield

Tabel IV menunjukkan hasil uji performa aplikasi DDoShield saat kami coba mengunggah file .pcap secara langsung ke server lokal kami.

Aplikasi DDoShield dapat menganalisis file .pcap kecil (100 flow) dengan waktu latensi hanya 6,08 ms, pcap sedang (1.000 flow) dalam 8,89 ms, dan pcap besar (10.000 flow)

Tabel II
PERFORMA XGBOOST PER VARIASI K PADA DATASET CIC-IDS2017

K	Accuracy (%)	Prec. (%)	Recall (%)	F1-Score (%)	AUC-ROC	Training (s)	Infer. (ms/flow)
5	99,88	≥99,8	≥99,8	99,89	0,9999	5,54	0,003
10	99,94	≥99,9	≥99,9	99,95	1,0000	3,41	0,002
15	99,96	≥99,9	≥99,9	99,96	1,0000	3,59	0,002
20	99,98	≥99,9	≥99,9	99,98	1,0000	3,88	0,003
30 *	99,9978	100,00	99,9961	99,9977	1,0000	4,44	0,005
all (78)	99,9978	100,00	99,9961	99,9977	1,0000	9,38	0,005

* = K optimal; Prec./Recall macro-average.

Tabel III
PERBANDINGAN XGBOOST VS. RANDOM FOREST (K=30)

Metrik	XGBoost	RF	Selisih
Accuracy (%)	99,9978	99,9934	+0,0044
Precision-DDoS (%)	100,0000	99,9961	+0,0039
Recall-DDoS (%)	99,9961	99,9922	+0,0039
F1-Score / macro (%)	99,9977	99,9932	+0,0045
AUC-ROC	1,0000	1,0000	0,0000
FPR (%)	0,0000	0,0051	-0,0051
Training Time (s)	4,44	30,74	-26,30
Inference (ms/flow)	0,0051	0,0084	-0,0033

Tabel IV
HASIL PENGUJIAN PERFORMA DDOSHIELD

Metrik	Nilai	Satuan	Kondisi
Latensi (100 flow)	6,08	ms	.pcap kecil
Latensi (1000 flow)	8,89	ms	.pcap sedang
Latensi (10000 flow)	56,69	ms	.pcap besar
Throughput	112.512,52	flow/s	.pcap sedang
CPU idle	17,1	%	Server idle
CPU aktif	61,0	%	1000 flow
Memory idle	152,9	MB	Server idle
Memory aktif	430,6	MB	1000 flow
FPR (BENIGN only)	0,00	%	100% normal

dalam 56,69 ms. Kecepatan pemrosesan inferensi *backend* (*model throughput*) mencapai puncak di angka 112.512,52 flow per detik (angka ini adalah kecepatan prediksi model di memori, tidak termasuk waktu tunggu ekstraksi *parsing* PCAP oleh Scapy). Saat server dalam kondisi diam (*idle*), memori RAM yang digunakan adalah 152,9 MB dengan beban CPU 17,1%. Ketika memproses file dengan 1.000 flow, memori RAM naik menjadi 430,6 MB dan beban CPU menjadi 61,0%. Hasil *False Positive Rate* (FPR) adalah 0,00%, yang menunjukkan bahwa aplikasi ini tidak salah mendeteksi data normal sebagai serangan.

Sebagai bukti validitas fungsional (*functional testing*), kami menguji aplikasi ini secara langsung menggunakan sampel berkas .pcap yang memuat jejak serangan jaringan. DDoShield terbukti mampu mengelompokkan paket-paket mentah menjadi aliran terstruktur (*flows*) secara otomatis dan berhasil melabeli aliran serangan secara tepat. Fitur forensik visual *TreeSHAP* pada dasbor DDoShield juga berhasil menyoroti anomali pada fitur Flow Packets/s dan Fwd Packet Length sebagai pemicu vonis DDoS. Temuan ini konsisten

dengan hasil analisis manual yang kami lakukan menggunakan Wireshark pada berkas yang sama. Hal ini menjadi bukti bahwa mekanisme *parsing* PCAP dan inferensi XGBoost dalam aplikasi DDoShield berjalan dengan benar dan dapat diandalkan.

V. PEMBAHASAN

A. Interpretasi Temuan Utama

Keberhasilan model XGBoost dalam mendeteksi DDoS dengan F1-Score mendekati 100% dikarenakan cara kerja algoritma *gradient boosting* yang memperbaiki kesalahan tebakan model sebelumnya di setiap pohon keputusan baru [14]. Selain itu, teknik regularisasi yang ada pada XGBoost juga sangat membantu mencegah model agar tidak terlalu menghafal data latih (*overfitting*) [13]. Akurasi yang sangat tinggi ini juga dilaporkan oleh peneliti lain pada dataset CIC-IDS2017 [19] karena kelas data pada dataset ini memang memiliki pola trafik yang sangat jelas dibedakan secara statistik.

B. Implikasi Praktis untuk Kampus

Karena proses pendeteksian XGBoost sangat cepat (0,0051 ms per flow), aplikasi DDoShield ini dapat diintegrasikan langsung pada router kampus untuk menganalisis salinan trafik (*port mirroring*) secara berkala. Aplikasi ini juga dilengkapi dengan REST API sehingga bisa dihubungkan ke sistem pemantauan keamanan lain yang sudah ada di kampus.

C. Validasi dan Keunggulan Dibandingkan Aplikasi Lain

Untuk memastikan program DDoShield berjalan dengan benar dan hasilnya valid, kami membandingkannya dengan dua aplikasi jaringan yang sudah umum digunakan, yaitu **Wireshark** dan **Snort**. Wireshark sering dijadikan standar acuan (*ground truth*) karena mampu menampilkan isi paket jaringan secara langsung. Namun, menganalisis ratusan ribu paket DDoS di Wireshark secara manual tentu membutuhkan waktu lama dan rawan terjadi kesalahan. Sementara itu, Snort adalah aplikasi pendeteksi penyusupan (IDS) yang cepat, tetapi karena berbasis aturan statis (*signature-based*), ia sering kesulitan mendeteksi serangan tipe baru (*zero-day*) yang belum ada aturannya.

DDoShield hadir untuk menutupi kekurangan kedua aplikasi tersebut. Seperti yang ditunjukkan pada Tabel V, DDoShield memberikan akurasi pendeteksian yang valid dengan waktu proses yang jauh lebih efisien dibandingkan analisis manual

Tabel V
PERBANDINGAN VALIDASI DDoSHIELD DENGAN APLIKASI JARINGAN UMUM

Kriteria	DDoShield (Kami)	Wireshark	Snort
Metode Analisis	Machine Learning	Inspeksi Manual	Aturan Statis (<i>Signature</i>)
Validitas Hasil	Sangat Tinggi (Berdasarkan Dataset)	Standar Acuan Utama	Bergantung <i>Update</i> Aturan
Deteksi Serangan Baru	Bisa (Melihat Pola Anomali)	Bisa (Tergantung Analisis)	Tidak Bisa
Kecepatan Deteksi	Sangat Cepat (0,0051 ms/flow)	Lambat (Manual)	Cepat (Otomatis)
Cara Analisis (Forensik)	Grafik Visual (<i>TreeSHAP</i>)	Baca Kode <i>Hex/Bytes</i>	Baca Teks Peringatan (<i>Alert</i>)

Wireshark. Berbeda dengan Snort yang kaku pada aturan, model XGBoost pada DDoShield berpotensi mengenali varian serangan yang belum memiliki *signature* karena klasifikasinya berbasis pola statistik aliran (*flow statistics*), bukan pencocokan teks paket. Meskipun demikian, kemampuan ini belum diuji secara eksplisit pada serangan *zero-day* dalam penelitian ini. Selain itu, untuk keperluan forensik, DDoShield menyediakan grafik *TreeSHAP* yang langsung menampilkan fitur penyebab serangan. Hal ini memudahkan pengguna untuk melakukan evaluasi tanpa harus menganalisis kode *byte hexadecimal* paket yang rumit seperti di Wireshark. Dengan demikian, program ini tidak hanya akurat secara komputasi, tetapi juga mudah dan praktis digunakan.

D. Keterbatasan

Adapun beberapa keterbatasan ruang lingkup dalam penelitian ini meliputi: (1) penggunaan dataset standar CIC-IDS2017 sebagai representasi trafik simulasi [19]; (2) meskipun DDoShield mendukung mode *Live Sniffing*, evaluasi performa pada penelitian ini difokuskan pada mode analisis berkas *.pcap* rekaman (*offline capture*); (3) uji coba performa aplikasi yang dibatasi pada satu komputer server lokal; (4) eksperimen dilakukan pada satu kali pembagian data (*single split*, *seed=42*) tanpa replikasi multi-seed, sehingga stabilitas varians hasil belum diuji secara statistik; dan (5) model belum dievaluasi terhadap jenis serangan siber terbaru yang menggunakan teknik manipulasi data (*adversarial attacks*).

E. Arah Penelitian Mendatang

Untuk penelitian selanjutnya, kami berencana untuk: (1) mengumpulkan data trafik langsung dari jaringan kampus kami; (2) mengembangkan sistem deteksi *real-time* dari trafik jaringan yang mengalir langsung (*live capture*); (3) menguji ketahanan model terhadap serangan yang dimanipulasi; dan (4) mencoba metode pembaruan model secara otomatis agar tetap akurat seiring berjalannya waktu (*continual learning*).

VI. KESIMPULAN

Dalam penelitian ini, kami telah mengembangkan dan mengevaluasi sistem deteksi DDoS berbasis XGBoost dengan seleksi fitur *Information Gain*. Kami juga membuat prototipe aplikasi web bernama DDoShield untuk membantu mendeteksi serangan di jaringan kampus secara lebih mudah.

Model XGBoost dengan $K = 30$ fitur terbaik berhasil mencapai *F1-Score* sebesar 99,9977% dan nilai AUC-ROC sebesar 1,0000. Dari 45.149 data uji, model kami hanya salah menebak

1 data saja, sedikit lebih baik dibandingkan dengan *Random Forest* (99,9932%). Metode seleksi fitur IG terbukti sangat membantu karena dapat memotong waktu pelatihan sebesar 52,7% tanpa menurunkan akurasi. Uji McNemar menunjukkan akurasi kedua model tidak berbeda secara signifikan secara statistik ($p = 0,4795$), namun XGBoost jauh lebih cepat dalam melatih data ($6,9\times$ lebih cepat) dan memproses flow ($1,6\times$ lebih cepat).

Hasil ini membuktikan bahwa kombinasi XGBoost dan seleksi fitur IG sangat cocok digunakan untuk mendeteksi DDoS di kampus yang memiliki keterbatasan sumber daya komputer. Di masa depan, kami berharap bisa menguji sistem ini menggunakan trafik kampus nyata secara langsung.

UCAPAN TERIMA KASIH

Para penulis mengucapkan terima kasih kepada Program Studi Rekayasa Keamanan Siber, Politeknik Siber dan Sandi Negara, atas dukungan akademik selama penelitian ini. Terima kasih juga kepada *Canadian Institute for Cybersecurity*, Universitas New Brunswick, atas ketersediaan dataset CIC-IDS2017 yang digunakan dalam penelitian ini.

PUSTAKA

- [1] NETSCOUT Systems, "NETSCOUT threat intelligence report," NETSCOUT Systems, Tech. Rep., 2023. [Online]. Available: <https://www.netscout.com/threatreport>
- [2] Cloudflare, "DDoS threat report Q3 2023," Cloudflare, Inc., Tech. Rep., 2023. [Online]. Available: <https://blog.cloudflare.com/ddos-threat-report-2023-q3>
- [3] G. B. A. Mthunzi, M. O. Adigun, and J. A. Ojo, "Adoption of BYOD in higher education: Security vulnerabilities and solutions," *Int. J. Comput. Theory Eng.*, vol. 12, no. 3, pp. 78–84, 2020.
- [4] M. N. Al-Mhiqani, R. Ahmad, and Z. Z. Abidin, "Cyber security challenges in higher education institutions: A systematic review," *Int. J. Adv. Comput. Sci. Appl.*, vol. 11, no. 7, pp. 125–135, 2020.
- [5] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Comput. Secur.*, vol. 86, pp. 147–167, 2019.
- [6] S. M. Kasongo and Y. Sun, "Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset," *J. Big Data*, vol. 7, no. 1, pp. 1–20, 2020.
- [7] I. Ullah and Q. H. Mahmoud, "A two-level flow-based anomalous activity detection system for IoT networks," *Electronics*, vol. 9, no. 3, p. 530, 2020.
- [8] O. H. Abdulganiyu, T. A. Tchakoucht, and Y. K. Saheed, "A systematic literature review for network intrusion detection system (IDS)," *Int. J. Inf. Secur.*, vol. 22, pp. 1125–1162, 2023.
- [9] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [10] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, p. 20, 2019.

- [11] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," vol. 7, 2019, pp. 41 525–41 550.
- [12] R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Applying convolutional neural network for network intrusion detection," pp. 1222–1228, 2017.
- [13] A. Thakkar and R. Lohiya, "A review of the advancement in intrusion detection datasets," *Procedia Comput. Sci.*, vol. 167, pp. 636–645, 2020.
- [14] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," pp. 785–794, 2016.
- [15] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.
- [16] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, 2003.
- [17] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," pp. 108–116, 2018.
- [18] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1705.07874>
- [19] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," pp. 7–12, 2021.